NanoBook: Phase 6 Report

Optimise: one measured change at a time · 24 October 2026 · C++23, GCC 14 / Clang 19, Linux (WSL2) · Phase 6 complete except the bare-metal re-checks

1. Summary

Phases 1–5 built a correct order book, a way to time it, a real Nasdaq day to feed it, and a three-thread pipeline around it. They left **symptoms**: a slow tail, a pipeline slower than one thread, a book much slower on real data than in cache. Phase 6 turned each symptom into an **experiment**: guess the cause, make the smallest change that tests the guess, time it against the old code, and keep it only if it wins by a rule written down before the timing.

Status	**Complete except the bare-metal re-checks** (§9): every experiment has a before/after table in `docs/perf-log.md`, the README's exit criterion
Experiments	**11 experiments**, **21 versions timed** in 28 A/B sessions, plus one closed on paper (E8)
Kept	**3**: the ring's back-off (E1b), the hybrid search (E3), huge pages (E6); plus E2, folded into E6
Not kept	**7 ideas**: 64-byte padding, LTO, `-march=native`, PGO, prefetching, the moving window (3 versions), batched ring 2 (3 versions)
Undecided	**1**: 8-byte ID-table entries (E9b), re-tested on bare metal
Whole day, one thread (WSL2, not for publishing)	about **108 → 92 ns per message** (15% faster), p99 about **437 → 390 ns** (§7)
Whole day, three threads	about **206 → 177 ns per message** (14% faster) from E1b alone (§7)
Correctness	Every kept change: the full real day identical to the reference book (0 mismatches in 363 million checks), the pipeline's 20 GB log byte-identical, 10^7+ fuzzed operations (§6)
Tests	**789** in Release builds (770 with sanitizers), up from 521 at the end of phase 5
Planted bugs	**34**, every one caught in the end; two first got past the unit tests and now have their own (§6.3)

The one-sentence lesson: several changes made the average faster and the slowest messages slower (PGO, the moving window, batched ring 2), and only a rule fixed in advance stopped them from being kept on their average alone.

2. What phase 6 is, in plain words

2.1 "Optimise" means "experiment"

A tempting way to optimise is to make every change that sounds faster. Phase 2 had already shown why not: a search trick that "had to be faster" was slower on every benchmark. Modern CPUs hide and amplify costs in ways that are hard to predict (caches, branch prediction, running instructions out of order), so **only a measurement decides**.

So each change is an experiment with four parts, written down before any code:

1. **Symptom:** a number already measured. "Three threads are slower than one."
2. **Hypothesis:** what causes it, and what number should move if that's right. "The two threads keep stealing one cache line from each other."
3. **Change:** the smallest edit that tests it. "Make one of them look at that line less often."
4. **Decision:** time old against new in one sitting; keep or revert by the rule (§3.3).

A reverted experiment is still a result. "We tried X, it didn't help, here is the data and here is why" is as useful as a win, and the perf log records both the same way.

2.2 A few words used throughout

- **Throughput:** how fast the whole day goes, in nanoseconds per message. Lower is better.
- **p50, p99:** the latency that 50% (or 99%) of messages beat. p50 is the typical message; p99 is the slowest 1 in 100, the **tail**. A trading system is judged on its tail.
- **Cache line:** the 64 bytes the CPU moves between memory and its caches in one go. If two cores keep writing the same line, it bounces between them (**ping-pong**), about 90 ns each way.
- **WSL2:** Linux running inside Windows, this laptop's set-up. It hides the CPU's hardware counters (cache misses, TLB misses), so every "because of the cache" explanation here stays a **hypothesis** until the bare-metal session (§9). Numbers from WSL2 are marked **not for publishing** (README rule 20).

---

3. How a change was judged

3.1 The A/B session

`scripts/run_ab_session.sh` builds the old code ("before", from git) and the new code ("after", the working tree), then runs them **alternately**: before, after, after, before, ... (ABBA).

Why alternate? A laptop warms up, turbo changes, Windows wakes up in the background. If all the "before" runs came first and all the "after" runs second, any drift in the machine would look like an effect of the change. Alternating spreads the drift over both sides equally.

Each session saves everything: every run's output, a progress log with times, the exact code change (`after.diff`), and both build logs. 28 sessions live under `docs/data/phase6_wsl/`.

3.2 Noise, and how big it was

The same binary run twice never gives the same number. This phase measured that noise many times:

- **Within one session:** the same "before" binary varied by **1–2.5%** in throughput.
- **Between sessions:** much more. The same code ran at **89–95 ns per message** one evening and **about 110 ns** the next night. One session was spoiled outright: halfway through, even parsing (code that didn't change) got 19% slower.

So, **only numbers from inside one session are compared**, and a gain smaller than that session's own spread is no gain.

3.3 The keep rule (fixed before phase 6 started)

A change is kept only if:

1. its **decision metric** (chosen before running) is better **in every round**;
2. the gain is **larger than the noise** (the round-to-round spread of either side);
3. **no guard metric** got worse beyond its noise;
4. every **correctness gate** passes (§6).

Example: for E7, the decision metric was throughput and the guards were p50, p99 and p99.99. E7 made throughput better in 3 of 4 rounds and p99 worse in all 4. It fails points 1 and 3.

A marginal case (a small gain for a lot of complexity, or one guard slightly worse) goes to the user to decide. That happened twice: E2 and E9b.

---

4. The experiments, simplest first

Each one: what it is, an example, what happened, and **so what**.

4.1 E5: ring padding, 64 against 128 bytes (a one-number change)

What: the ring buffer keeps its four counters 128 bytes apart, not 64, on the README's advice: Intel CPUs often fetch cache lines in pairs, so two counters 64 bytes apart can still interfere. It had never been measured.

Example: like two people with desks side by side; even if each uses only their own desk, the cleaner moves both desks together, so one person's work disturbs the other's.

Result: with 64 bytes, a message's round trip between two cores was **slower in all 4 rounds**; the full pipeline didn't notice. **Not kept: 128 stays.**

So what: the rule we followed blindly turned out right on this machine. Also new: WSL2's core pinning is unreliable (the same binary ran at 5 or at 20 ns per record), so every between-core number waits for bare metal.

4.2 E4: compiler options (no code changed at all)

What: three ways to make the compiler try harder:

- **LTO** (link-time optimisation): optimise across source files.
- `-march=native`: use every instruction this exact CPU has.
- **PGO** (profile-guided optimisation): run the program once with counters, then recompile using which paths really ran most.

Example: PGO is like a shop rearranging its shelves so the best-sellers sit by the door. Faster for most customers; slower for anyone who wants something that got moved to the back.

Result:

Option	Throughput	The catch
LTO	no gain	Expected: the book is all headers, so the compiler already sees it all at once
`-march=native`	**1.8% slower**	Parsing 25% slower: new instructions aren't automatically faster
PGO	**1.4% faster**	**Replaces 25% slower** at p50, p99 3–5% worse

None kept as the default; all three stay as build presets for the bare-metal re-check.

So what: "make the compiler try harder" isn't free. PGO is the first example in this phase of **a better average bought with a worse tail**, and the rule caught it.

4.3 E2: page faults (touch memory before the clock starts)

What: when a program first writes to a 4 KiB page of memory, the operating system has to step in (a **page fault**, a few µs). The books' arrays were only reserved at start-up, so their pages were first touched in the middle of the timed replay.

Example: like a hotel that only makes up a room when the guest is already at the door.

Change: write once to every page at start-up.

Result: page faults while timed **4,758 → 0**; p99.99 about **12% better**; p99 about 2% worse (maybe real, maybe code moving around). **Kept provisionally** (the user's decision), with a bare-metal re-check whose criteria were written down before the data (B5-E2). E6 later absorbed it.

So what: the fault count alone was 8× too small to explain the slow tail, yet removing the faults helped more than the count predicts. A clue that a fault also costs the messages after it (the kernel evicts cache lines), to check on bare metal.

4.4 E6: huge pages (fewer, bigger pages)

What: the CPU translates every memory address through a small table of recently used pages (the **TLB**). The books' 384 MB is about 100,000 pages of 4 KiB, far more than the TLB holds, so many messages wait for a slow translation. With 2 MiB **huge pages**, it is about 200 pages.

Example: a street directory with 100,000 small entries you keep having to look up, against one with 200 entries that fits on a single card in your pocket.

First, a test with no code: glibc can be told to use huge pages with an environment variable. Result: **4.8% faster**. The hypothesis held, so it was worth building properly.

Change: `support/huge_page_arena.hpp`, one 2 MiB-aligned region, all its pages touched at start-up (E2's job, done once), with standard C++ `std::pmr` containers taking their memory from it.

Result: 5.6% more throughput, p50 5% and p99 11% better, in both rounds. **Kept.**

So what: a real, sizeable win from changing where memory lives, without touching the book's logic. But a virtual machine translates every address twice, which may exaggerate it: the bare-metal re-check is mandatory (B5-E6).

4.5 E3: the hybrid search (real data overturns a synthetic result)

What: to find a price among a book's levels, the book used a **binary search** (look in the middle, halve, repeat: about 12 steps for thousands of levels). The hybrid first checks the **4 levels nearest the best price** one by one.

Example: looking for a word in a dictionary by halving, against first glancing at the page you already have open, because you were just there.

History: phase 2 dropped the hybrid: on synthetic orders spread evenly over 100 levels, it was slower. But the real day (phase 4) showed **48% of new orders land at or better than the best price**, in books thousands of levels deep.

Result: K = 4 gave **10% more throughput** in both rounds, every guard equal or better. K = 8 and 16 were no faster and slowed adds. **Kept: K = 4.**

So what: a benchmark is only as good as the data it runs on. The same idea that lost on made-up data won on real data, because real orders cluster near the best price.

4.6 E1: the pipeline's ping-pong (E1, E1c, E1d)

Symptom (phase 5): the whole day ran at 174 ns per message on one thread but **237 on three**. Splitting the work made it slower.

Hypothesis: the feed thread is faster than the engine, so ring 1 is always full. The feed keeps re-reading the engine's "how far I've read" counter (`tail_`), pulling its cache line away, and every time the engine moves on it has to pull the line back. That's ping-pong.

Example: two people sharing one notebook. One keeps grabbing it to check a number; the other needs it back to write the next number. Most of the time goes on passing the notebook.

E1, two opposite cures, both measured:

- **(a) Batching:** the engine updates its counter every 64 records, not every record.
- **(b) Back-off:** the feed, finding the ring full, waits a little longer before looking again.

Variant (each in its own session)	Three threads before, ns per message	After	Gain
(a) batching	206.7 / 205.4	179.0 / 169.7	15.4%
(b) back-off	205.7 / 202.7	176.8 / 177.0	13.4%
(a) + (b)	220.5 / 213.9 (a warmer evening)	182.0 / 181.2	16.4%

Two opposite cures giving the same gain is strong evidence the hypothesis is right. **Kept: (b)**, the simplest (the user's choice). Paced latency below capacity was unchanged.

E1c, does the pipeline pay at all? One thread against three, at a steady rate:

- At 1–2 M messages/s, **one thread is faster**: p50 about 205 ns against 500. Three threads pay two hand-overs between cores (about 135 ns) that one thread never does.
- Near capacity, **three threads hold up better**: they sustain 4.9 M/s or more, one thread 4.3.

Both modes stay; which is right depends on how much work sits around the book.

E1d, the same ping-pong on ring 2 (engine → publisher). The engine publishes ring 2's counter in batches. Three versions (batch 64, batch 8, and "publish first whenever no input is known to be waiting"):

- Three threads **7–12% faster in every round**, now matching one thread for the first time.
- But at a steady 2 M/s, **p90 got worse** (in every round, or 3 of 4 for the last version). The feed arrives in bursts; during a burst the engine never pauses, so its updates wait for their batch.

Not kept.

So what: the same cure helped ring 1 and hurt ring 2, for a clear reason. On ring 1, batching delays free slots that a waiting feed can afford to wait for. On ring 2 it delays updates, which are the output whose latency we measure. **Where a delay lands decides whether it's free.**

4.7 The profile, and E9: the ID table

Before the biggest experiment (E7), a **profile** checked what it could gain. `perf record` interrupted the replay about 2,000 times a second and noted where it was each time.

Part of the book	Share of the replay
ID table (find an order by its id)	**50%**, 39% on one line: comparing ids
The order's own record, first read	12%
Price levels (E7's target)	**23%**
Everything else	about 15%

The hot spots were the places where the CPU **waits for memory**. The ID tables total over 100 MB against 30 MB of the CPU's largest cache, so a delete, an execution or a replace usually waits twice: once for the id's table entry, then for the order.

The user's decision: attack the ID table first (E9), since it is twice the size of E7's target.

- **E9a, prefetch:** while handling one message, ask the CPU to start fetching the next message's table entry. Result: **6% slower**. The extra work on every message cost more than the overlap saved. **Reverted.**
- **E9b, smaller entries:** 16 → 8 bytes each (a 4-byte "tag" of the id, confirmed against the order itself), so twice as many fit in cache. Lookups got clearly faster (executions 18–30%, deletes 6–24% at p50), but throughput was better in 2 rounds and worse in 2, in two unusually noisy sessions. **Undecided: reverted for now, re-tested on bare metal (B6).**

So what: measure before redesigning. The profile moved the biggest target from levels to the ID table, and set a hard ceiling on what E7 could ever win (23%).

4.8 E7: the moving window (the biggest experiment)

What: a **tick array** gives every possible price its own slot, so finding a price needs no search at all. Phase 2 measured it 1.2–4× faster than the sorted list. But a real day's prices run from $0.0001 to $200,000 (traders park silly "stub" orders far away), far too many slots.

The idea: slots only for the prices **near the best**, a sorted list for the rest, and the slots slide along when the price moves.

Example: lockers numbered by price for the 1,024 prices around the current price; the rare faraway prices on a shelf at the back.

How it was built (each step with its own checks):

1. **A count before any code** (`support/itch_window.hpp`): a pretend window on the real day. With 1,024 slots, **98.3% of lookups** land in a slot, and the slots slide about once every 390 messages, carrying 2–7 levels. 256 slots reached only 93.7%, below the 95% set in advance.
2. **The real structure** (`WindowLevels<W>`): slots arranged like a **clock face**, price p in slot (p ÷ step) mod 1,024, so when the window slides, prices that stay keep their slot.
3. **Proof:** the full day against the reference book, **0 mismatches** in 363 million checks.

Result, three timed sessions:

	Throughput	p99
E7	7–13% better in 3 rounds, 9% worse in 1	**10–53% worse in every round**
E7b(a): a slot holds its own price	1–2% better	**21–26% worse**

	Throughput	p99
E7b(a+b): plus each side remembers its best	3–5% better	**18–22% worse**

Even **adds** got a worse tail, and adds never search for anything. So the cost is the window's **memory layout**: empty slots between prices spread the same levels over more cache lines than the sorted list, which keeps its levels near the best packed at one end. A hypothesis for bare-metal cache-miss counters (B7). **Not kept:** the window stays in the code as a tested experiment, like phase 2's tick array.

So what: the window really did what it promised on average (level work fell from 23% to 16% of the replay), and still lost, because **an average can improve while the tail gets worse**. Three sessions and two targeted fixes made that conclusion solid rather than a bad night.

4.9 E8: deep inserts (closed on paper)

What: inserting a new price deep in a big book shifts every better level along by one. The real day has a few such inserts (0.02% shift 1,024 levels or more).

Result without timing anything: from the day's own counts, deep inserts are **at most about 0.16% of the replay**, ten times smaller than one session's noise. **Closed without an experiment.**

So what: an honest bound can be a result. Timing something you can't detect only produces a number that looks like evidence.

---

5. All experiments at a glance

#	Experiment	Kind	Decision metric	Result	Decision
E5	Ring padding 64 vs 128 B	one constant	ring round trip	64 B worse in 4/4	**not kept**
E4	LTO	build flag	throughput	no gain	**not kept**
E4	`-march=native`	build flag	throughput	1.8% slower	**not kept**
E4	PGO	build	throughput	1.4% faster, replaces 25% slower	**not kept**
E2	Touch level pages at start-up	memory	p99.99	12% better, p99 2% worse	**kept** (in E6)
E6	Huge-page arena	memory	throughput	5.6% faster, p99 11% better	**kept**
E3	Hybrid search, K = 4	algorithm	throughput	10% faster	**kept**
E1	Ring 1 back-off	threads	three-thread ns/msg	13–16% faster	**kept** (b)
E1c	One thread vs three	question	latency	one wins at normal load	both modes stay
E1d	Batched ring 2 (3 versions)	threads	three-thread ns/msg	7–12% faster, p90 worse	**not kept**
E9a	Prefetch ID entry	memory	throughput	6% slower	**not kept**
E9b	8-byte ID entries	data layout	throughput	lookups faster, throughput mixed	**bare metal** (B6)
E7	Moving window (3 versions)	data structure	throughput	1–13% faster, p99 18–53% worse	**not kept**
E8	Deep inserts	bound	–	≤ 0.16% of the replay	**closed on paper**

---

6. How correctness was kept

Speed is worthless if the answers change. Every change passed these **gates** before its timing counted (optimise design §4):

6.1 The same answers

- **The full real day, reference book against fast book:** after every one of 363,118,215 order messages, the two books agree (a quick check every message, a full check of every level and order 61,354 times). Run for E2, E3, PGO, E6, E9b, E7 and E7b.
- **The pipeline's log:** the full day through three threads writes a 20 GB log **byte-identical** to one thread's, checked by one checksum (`748abedf40533b82`). Run for E1, E5, E6, E9b, E1d.
- **The fuzzer:** 10 million random operations or more on several seeds, fast book against reference book, for every change to a book.

6.2 No new costs on the hot path

- **Zero allocations** after start-up, tested for every book and the pipeline.
- **ThreadSanitizer** clean for every ring change, also squeezed onto 2 CPUs.
- **Machine code unchanged where it should be:** for E3 (5,527 lines of assembly identical with the search turned off) and for E7's additions to the sorted list (identical apart from label numbers).

6.3 Planted bugs: do the tests have teeth?

For every change with new logic, a deliberate bug was put in, and a test had to fail. **34 planted bugs, every one caught in the end.** Three examples, simplest first:

- **E3:** an off-by-one in the search: the tests and the fuzzer both crashed.
- **E1d:** forgetting to publish before waiting on a full ring: the test hung, which is exactly the deadlock it would cause in production.
- **E1a:** publishing a counter with `relaxed` instead of `release` memory order: only ThreadSanitizer noticed.

Two first got past the unit tests, and each now has its own test:

- **The tick array's bitmap** (since phase 2): when a 64-slot block emptied, forgetting to clear its summary bit passed every unit test; only the fuzzer crashed.
- **E7b:** a slot keeping a stale price after a slide: only the fuzzer caught it.

So what: the fuzzer earned its keep twice, and the gap it found in phase 2's tests had been there all along. A planted bug that no test catches is the most useful outcome: it shows where the tests are blind.

6.4 Tests

789 in Release builds (770 with sanitizers), on GCC and Clang, in CI on every push: up from 521 at the end of phase 5. Most of the new ones are the existing typed tests now also running on the two window books.

---

7. What the kept changes add up to

All WSL2, not for publishing. Totals across sessions are rough: the machine drifted by up to 20% between sessions (§3.2).

	Start of phase 6	After E3 and E6	Change
Whole day, one thread, ns per message	108.4 (E2's "before")	91.5–92.5 (E6's "after")	about **15% faster**
p99, all order messages	437–441 ns	390–392 ns	about **11% better**
Page faults while timed	4,758	0	**gone**
Pipeline, three threads, ns per message	about 206 (E1's "before")	about 177 (E1b)	about **14% faster**

So what: about 15% faster on one thread and 14% on three, with a better tail, from three small changes (one constant, one search tweak, one memory arena). The big redesign (E7) gave nothing kept. On this workload the cheap, well-aimed changes won.

---

8. What phase 6 taught

1. **Measure before believing.** "Obviously faster" was wrong for `-march=native`, for prefetching and for the moving window, and "obviously slower" (the hybrid search) was wrong too.
2. **Data beats intuition, and real data beats made-up data.** The hybrid search lost on synthetic orders and won by 10% on the real day.
3. **Averages hide tails.** PGO, the moving window and batched ring 2 all improved the average and hurt the slow messages. Without guards fixed in advance, all three would have been kept.
4. **Profile before redesigning.** The profile moved the main target (the ID table, 50%) and capped E7's possible gain (23%) before a line of it was written.
5. **Where a delay lands matters.** Batching a counter was free when it delayed free slots (ring 1) and costly when it delayed the output (ring 2).
6. **Noise is the enemy, and it has to be measured.** One session was spoiled mid-run; the same binary ran 20% apart on different evenings. Alternating runs, several rounds, and comparing only inside a session made the results trustworthy.
7. **Negative results are results.** Seven ideas not kept, each with its data and a reason, say as much about this machine and this workload as the three kept.
8. **A bound can replace an experiment.** E8 was closed with arithmetic, because no timing could have detected it.

---

9. What waits for bare metal

WSL2 can't count cache misses or TLB misses, and its timing between cores is unreliable. One bare-metal session (Ubuntu from a USB stick, isolated cores) will run these, each with its criteria written down already (optimise design §5):

#	What	Why
B1	Cache and TLB misses per message on the real day	Turns this report's "hypotheses" (E2, E3, E6, E7) into facts or refutes them
B2	`perf c2c` on the pipeline	Shows the cache-line ping-pong of E1 and E5 directly
B4	Tails on isolated cores	Separates the machine's stalls from the code's
B5-E2, B5-E6	Re-run the kept memory changes	A VM translates addresses twice, which may flatter them. Revert if the criteria fail
B6	E9b's 8-byte ID entries	Undecided on WSL2: bare metal's lower noise decides
B7	E7's tail with cache-miss counters	Tests the "sparse window, more cache lines" explanation

The same session publishes phases 3, 4 and 5's numbers.

---

10. What's next

1. **Mark phase 6 complete** in the README (its exit criterion is met), with this report.
2. **The bare-metal session** (§9): phases 3–5's published numbers and phase 6's re-checks, from one USB-stick Ubuntu session. It waits only for the stick.
3. **Phase 7, polish** (README roadmap): the README's results with plots, design docs tidied, flame graphs, and an **interview-notes** section on the trade-offs made. Its exit criterion: a stranger can build, test and reproduce the numbers in under 10 minutes. This report's §8 is a first draft of those notes.